# Kubernetes: Arquitetura e Componentes

Orquestração de Aplicações em Contêineres

**Prof. Alexandre Huff**

alexandrehuff@utfpr.edu.br

Universidade Tecnológica Federal do Paraná (UTFPR)
Campus Toledo / SAET 2026

13 a 16 de outubro de 2026

UTFPR
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

# Conteúdo

Conceitos Arquiteturais

Workloads

Networking

Storage

Configuration

Auth & Identity

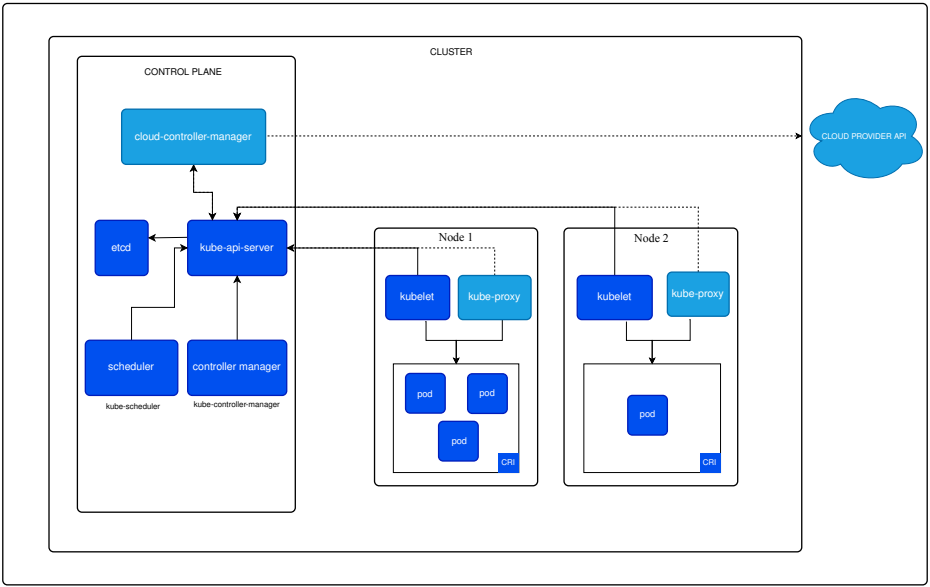
Atividades

# Conceitos Arquiteturais

# Conceitos Arquiteturais do Kubernetes

- Um cluster Kubernetes consiste em um **plano de controle** e um **conjunto de máquinas de trabalho**, chamadas nós, que executam aplicações em contêineres.
- Todo **cluster precisa** de pelo menos **um nó de trabalho** para executar os Pods.
- Os nós de trabalho (*Worker Nodes* ou somente *Nodes*) hospedam os Pods, que são os componentes da carga de trabalho da aplicação.
- O plano de controle gerencia os *Nodes* e os Pods no cluster.
- Em ambientes de produção, o plano de controle geralmente é executado em vários computadores (*Master Nodes*).
  - Um cluster geralmente executa vários nós, proporcionando tolerância a falhas e alta disponibilidade.

# Arquitetura de Referência do Kubernetes



# Componentes do Plano de Controle

- kube-api-server
  - Expõe a API do plano de controle do Kubernetes.
  - Fornece uma **interface REST** para **executar comandos** e obter **informações do cluster**, através da qual todos os outros componentes interagem.
- etcd
  - Fornece persistência de suporte do Kubernetes para todos os dados do cluster.
  - Utiliza armazenamento de chave-valor consistente e altamente disponível.
- kube-scheduler
  - Monitora a criação de novos Pods que ainda não foram atribuídos a um nó.
  - Seleciona um ou mais nós para a execução dos Pods.

# Componentes do Plano de Controle

- kube-controller-manager
  - Componente que gerencia todo o loop de controle dos principais componentes do cluster.
  - Monitora o estado do cluster via *kube-api-server* e “conduz” o cluster para o “estado desejado”.
  - Existem diferentes tipos diferentes de controladores. Alguns exemplos são:
    - **Node controller:** Responsável por detectar e responder quando os nós ficam inativos.
    - **Job controller:** Monitora *Job objects* que representam tarefas que são executadas pontualmente, em seguida, cria Pods para executar essas tarefas até a conclusão.
    - **EndpointSlice controller:** Preenche objetos de EndpointSlice (para fornecer um link entre Serviços e Pods).
    - **ServiceAccount controller:** Cria contas de serviço padrão para novos namespaces.
- cloud-controller-manager (opcional)
  - Componente que gerencia toda a lógica de integração com plataformas de nuvem.
  - Apenas executa controladores específicos para a plataforma de nuvem em uso.
  - Se o Kubernetes é executado em hardware local, este componente não é necessário.

# Componentes dos Nodes

São executadosem todos os nós. Mantém os Pods em execução provendo o ambiente de execução de aplicações do Kubernetes.

- kubelet
  - É um agente executado em cada nó do cluster.
  - Garante que os contêineres estejam sendo executados em um Pod.
- kube-proxy (opcional)
  - É um proxy de rede executado em cada nó do cluster, implementando parte do conceito de *Service* do Kubernetes.
  - Mantém regras de rede nos nós e permitem a comunicação em rede com Pods.
  - Utiliza a camada de filtragem de pacotes do sistema operacional, se disponível. Caso contrário, o kube-proxy encaminha o tráfego por conta própria.
  - Não é necessário executar o kube-proxy se for utilizado um plugin de rede que implementa o encaminhamento de pacotes.
- Runtime de contêineres
  - Componente fundamental que permite gerenciar a execução e o ciclo de vida dos contêineres no ambiente Kubernetes.

# Conceitos Básicos 🙄

- **Cluster:** Coleção de nodos que agregam recursos de CPU, RAM, Disco e dispositivos em um único grupo.
- **Objects:** São entidades persistentes usadas pelo Kubernetes para representar o estado do cluster.
- **Namespace:** Agrupamento lógico de recursos/objetos no cluster, também usado para definir escopos de permissões de acesso.

- **Metadata:** Dados que permitem identificar unicamente um objeto, incluindo *name*, *UID* e *namespace* (opcional).
- **Label:** Pares de chave-valor usado para identificar um grupo relacionado de objetos.
- **Annotation:** Pares de chave-valor usados para armazenar informações diversas e metadados.
- **Selector:** Utilizados para filtrar ou selecionar objetos por meio de *labels*.

# Conceitos Básicos 🙄

Labels, Annotations e Selector

```yaml
labels:
  app: nginx
  tier: frontend

annotations:
  description: "nginx frontend"

selector:
  matchLabels:
    app: nginx
    tier: frondend
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  annotations:
    description: "nginx frontend"
  labels:
    app: nginx
    tier: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      tier: frondend
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

# Workloads

# Workloads

- Um Workload (carga de trabalho) é uma aplicação executada no Kubernetes.
- Pode ser um único componente ou vários que trabalham em conjunto.
- Um Workload é executado por um conjunto de pods.

- No Kubernetes, um Pod representa um conjunto de um ou mais contêineres em execução no cluster.

# Workloads

O Kubernetes fornece diversos recursos de carga de trabalho integrados:

- Deployment
- ReplicaSet
- DaemonSet
- Job
- CronJob

É possível estender o Kubernetes e criar os próprios recursos de carga de trabalho utilizando *custom resource definition*.

Exemplo: Uma aplicação de alto desempenho que precisa de múltiplas réplicas de Pod em execução. Ou todos devem estar executando, ou nenhum deles!

# Pods

- É a menor unidade computacional (objeto) implantável no Kubernetes.
- Compreende um ou mais contêineres que compartilham o mesmo armazenamento, rede seu contexto (namespaces, cgroups, etc.).
- Geralmente é criado em função de outros recursos que dependem de Pods.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

Para criar o pod acima execute o comando:

```
$ kubectl apply -f https://k8s.io/examples/pods/simple-pod.yaml
```

# ReplicaSet

- Utilizado para garantir uma determinada quantidade de Pods em execução.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
      - name: php-redis
        image: us-docker.pkg.dev/google-samples/containers/gke/gb-frontend:v5
```

Para criar o Replicaset acima execute o comando:

```
$ kubectl apply -f https://kubernetes.io/examples/controllers/frontend.yaml
```

# Statefulset

- Gerencia o deployment e a escalabilidade de um conjunto de Pods.
- Executa um grupo de Pods e mantém identidades persistentes para cada um desses Pods.
- Mantém a identidade persistente e ordenada os Pods (`pod01`, `pod02`, ...).
- Cria volumes de persistência independentes para cada Pod baseado em `volumeClaimTemplates`
- Se usar volumes de persistência, é possível usar o `Statefulset` para tolerância à falhas.
- Se um Pod falhar, os identificadores persistentes de Pod permitem que os novos Pods criados sejam associados ao mesmo volume.

Para criar o Statefulset do slide seguinte execute o comando:

```
$ kubectl apply -f https://k8s.io/examples/application/web/web.yaml
```

# Statefulset

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None
  selector:
    app: nginx
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: registry.k8s.io/nginx-slim:0.21
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 1Gi
```

# DaemonSet

- Executa um Pod em cada nodo do cluster
- Alguns usos típicos de um DaemonSet são:
  - Executar um daemon de armazenamento em cada nó;
  - Executar um daemon de coleta de logs em cada nó;
  - Executar um daemon de monitoramento de nós em cada nó.
- Para criar o DaemonSet execute o comando:

```
$ kubectl apply -f https://k8s.io/examples/controllers/daemonset.yaml
```

# Deployments

- Cria ReplicaSets e Pods conforme especificados
- Permite que os recursos criados sejam gerenciados pelo cluster

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Para criar o Deployment acima execute o comando:

```
$ kubectl apply -f https://k8s.io/examples/controllers/nginx-deployment.yaml
```

# Networking

# Service

- Define a abstração de um serviço de rede.
- Permite adicionar uma porta local onde o *proxy* deve ouvir (*Listening*).
- Configura quais Pods devem responder à uma solicitação de serviço.
- Utiliza *labels* para definir quais Pods deve responder às solicitações.
- `port`: Define a porta do serviço.
- `targetPort`: Porta do contêiner.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Mais informações em:
`https://kubernetes.io/docs/concepts/services-networking/service/`

# Port Definitions

- As definições de porta nos Pods possuem o atributo `name`.
- É possível referenciar os nomes das portas dos Pods dentro da configuração de um `Service` pelo atributo `targetPort`.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app.kubernetes.io/name: proxy
spec:
  containers:
  - name: nginx
    image: nginx:stable
    ports:
      - containerPort: 80
        name: http-web-svc
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app.kubernetes.io/name: proxy
  ports:
  - name: name-of-service-port
    protocol: TCP
    port: 80
    targetPort: http-web-svc
```

# Multiport Services

- Para alguns serviços, é necessário expor mais de uma porta.
- O Kubernetes permite configurar várias definições de porta em um `Service`.
- Ao usar várias portas para um serviço, deve-se nomear todas as portas de forma que não haja ambiguidade.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
    - name: https
      protocol: TCP
      port: 443
      targetPort: 9377
```

# Service Types

Os Service Types do Kubernetes permitem especificar o tipo de serviço desejado.

Os valores são configurados no atributo `type` e seus comportamentos são:

- `ClusterIP`
  - Expõe o serviço em um IP interno do cluster.
  - Torna o serviço acessível apenas de dentro do cluster.
  - Esse é o padrão usado se não especificar explicitamente um tipo para o serviço.
  - É possível expor o serviço à internet pública usando o Ingress ou Gateway API.
- `NodePort`
  - Expõe o serviço no endereço IP de cada nó em uma porta estática (NodePort).
  - Para disponibilizar a porta do nó, o Kubernetes configura um endereço IP do cluster, da mesma forma que se tivesse solicitado um serviço do tipo `ClusterIP`.

# Service Types

- `LoadBalancer`
  - Expõe o serviço externamente usando um balanceador de carga externo.
  - O Kubernetes não oferece um componente de balanceamento de carga diretamente.
  - É necessário fornecer um balanceador de carga, ou, pode-se integrar o cluster Kubernetes a um provedor de nuvem.
- `ExternalName`
  - Mapeia o serviço para o conteúdo do atributo `externalName` (por exemplo, para o nome do host definido no `externalName`).
  - O mapeamento configura o servidor DNS do cluster para retornar um registro CNAME com esse valor definido em `externalName`, que geralmente é um endereço externo ao cluster.
  - Nenhum tipo de proxy é configurado.

# Exemplos de Service Types

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
    - nodePort: 30080  # Clientes externos acessam http://<Node-IP>:30080
      port: 80         # Pods internos acessam http://web-service:80
      targetPort: 8080 # Trafego é direcionado ao web-app na porta 8080
```

# Exemplos de Service Types

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  clusterIP: 10.0.171.239
  type: LoadBalancer
status:
  loadBalancer:
    ingress:
    - ip: 192.0.2.127
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

# Descoberta de Serviços

O Kubernetes suporta dois modos de descoberta de serviços: DNS e Variáveis de Ambiente

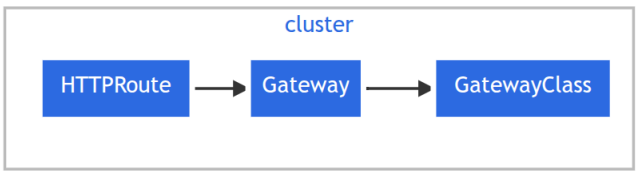
- DNS
  - Um servidor DNS, como o **CoreDNS**, monitora a API do Kubernetes em busca de novos serviços e cria um conjunto de registros DNS para cada um.
  - Se o DNS estiver habilitado em todo o cluster, todos os pods poderão resolver automaticamente os serviços pelo seu nome DNS.
  - Por exemplo, com um serviço chamado `my-service` em um namespace do Kubernetes chamado `my-ns`, será criado um registro DNS para `my-service.my-ns`. Os pods no namespace `my-ns` poderiam encontrar o serviço fazendo uma pesquisa pelo nome `my-service` (`my-service.my-ns` também funcionaria).
- Variáveis de Ambiente
  - Quando um Pod é executado, o `kubelet` adiciona um conjunto de variáveis de ambiente para cada serviço ativo.
  - As variáveis de ambiente SVCNAME_SERVICE_HOST e SVCNAME_SERVICE_PORT, de forma que o nome do serviço é definido em maiúsculas e os hífens são convertidos para *underlines*.

Por exemplo, o serviço **redis-primary**, que expõe a porta **TCP 6379** e ao qual foi atribuído o endereço IP de cluster **10.0.0.11**, produz as seguintes variáveis de ambiente:

```
REDIS_PRIMARY_SERVICE_HOST=10.0.0.11
REDIS_PRIMARY_SERVICE_PORT=6379
REDIS_PRIMARY_PORT=tcp://10.0.0.11:6379
REDIS_PRIMARY_PORT_6379_TCP=tcp://10.0.0.11:6379
REDIS_PRIMARY_PORT_6379_TCP_PROTO=tcp
REDIS_PRIMARY_PORT_6379_TCP_PORT=6379
REDIS_PRIMARY_PORT_6379_TCP_ADDR=10.0.0.11
```

# Gateway API

- É uma família de APIs que oferece provisionamento dinâmico de infraestrutura e roteamento de tráfego avançado.
- Disponibiliza serviços de rede utilizando um mecanismo de configuração extensível e compatível com protocolos de rede.
- Possui diferentes tipos de API:
  - **GatewayClass:** Define um conjunto de gateways com configuração comum e gerenciados por um mesmo controlador que implementa a GatewayClass.
  - **Gateway:** Define uma instância de infraestrutura de gerenciamento de tráfego, como por exemplo um balanceador de carga na nuvem.
  - **HTTPRoute e GRPCRoute:** Define regras específicas do HTTP para mapear o tráfego de um *listener* de `Gateway` para uma representação de *endpoints* de backend. Esses endpoints são frequentemente representados por `Service`.




Relações entre os três tipos de Gateway API.

# GatewayClass

- Os gateways podem ser implementados por diferentes controladores com configurações distintas.
- Um `Gateway` deve referenciar uma classe `GatewayClass` que tenha o nome do controlador que implementa a classe.
- Um exemplo mínimo de `GatewayClass`:

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: example-class
spec:
  controllerName: example.com/gateway-controller
```

Referência completa do tipo GatewayClass

# Gateway

- Define um endpoint de rede que pode processar tráfego, ou seja, filtrar, balancear, dividir, etc., e encaminhar para *backends* como um `Service`.
- Por exemplo, um `Gateway` pode representar um balanceador de carga na nuvem ou um servidor proxy em cluster configurado para aceitar tráfego HTTP.
- Neste exemplo uma instância de Gateway é definido para processar o tráfego HTTP na porta 80.
- O `hostname` é usado como um endpoint para processar o tráfego de serviços de *backend*.

Referência completa do tipo Gateway

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
  namespace: example-namespace
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    hostname: "www.example.com"
    allowedRoutes:
      namespaces:
        from: Same
```

# HTTPRoute

- Especifica o roteamento de requisições HTTP de um *listener* de `Gateway` para endpoints de rede de *backend*.
- Neste exemplo, o tráfego HTTP do `Gateway` “example-gateway” com o cabeçalho `Host:` definido como `www.example.com` e o recurso especificado como `/login` será roteado para o serviço `example-svc` na porta 8080 do mesmo namespace.

Referência completa do tipo HTTPRoute

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-httproute
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "www.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /login
    backendRefs:
    - name: example-svc
      port: 8080
```

# GRPCRoute

- Especifica o roteamento de requisições gRPC de um listener de `Gateway` para endpoints de rede de *backend*.
- Os gateways que suportam GRPCRoute devem suportar HTTP/2 sem uma atualização inicial de HTTP/1, garantindo assim o fluxo correto do tráfego gRPC.
- Neste exemplo, o tráfego gRPC do `Gateway` “example-gateway” com o host definido como `svc.example.com` será direcionado para o serviço `example-svc` na porta 50051 do mesmo namespace.

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GRPCRoute
metadata:
  name: example-grpcroute
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "svc.example.com"
  rules:
  - backendRefs:
    - name: example-svc
      port: 50051
```

Referência completa do tipo GRPCRoute

# Exemplo de Fluxo de Requisição



# Atividade Prática I

Instalar e configurar a Gateway API no MicroK8s utilizando o controlador Envoy Gateway configurado para expor portas via `NodePort`

1. Instalar a Gateway API e o Envoy Gateway via Manifesto
   - `$ kubectl apply --server-side -f https://github.com/envoyproxy/gateway/releases/download/v1.2.6/install.yaml`
   - Verificar se o pod do namespace `envoy-gateway-system` está com o estado `Running`
2. Configurar o modo `NodePort` via recurso `EnvoyProxy`
   - Por padrão, o Envoy Gateway tenta criar serviços do tipo LoadBalancer. Mas vamos utilizar o modo `NodePort` por causa da restrição local de endereços IPs.

# Atividade Prática II

```yaml
apiVersion: gateway.envoyproxy.io/v1alpha1
kind: EnvoyProxy
metadata:
  name: envoy-nodeport-config
  namespace: envoy-gateway-system
spec:
  provider:
    type: Kubernetes
    kubernetes:
      envoyService:
        type: NodePort
        patch:
          type: StrategicMerge
          value:
            spec:
              ports:
                - name: http
```

```
      port: 80
      nodePort: 31080
```

- Executar `microk8s kubectl -n envoy-gateway-system get envoyproxy` para confirmar que o recurso `envoy-nodeport-config` foi registrado com sucesso.

3. Criar a GatewayClass associada ao EnvoyProxy

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: eg-nodeport
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
  parametersRef:
    group: gateway.envoyproxy.io
    kind: EnvoyProxy
    name: envoy-nodeport-config
    namespace: envoy-gateway-system
```

- Executar `kubectl get gatewayclass eg-nodeport` e verifique se a coluna ACCEPTED exibe True.

4. Instanciar o Gateway e identificar a porta NodePort

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: meu-gateway
  namespace: default
spec:
  gatewayClassName: eg-nodeport
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
```

```
          from: Same
```

- Execute `kubectl -n envoy-gateway-system get svc` e identifique o serviço gerado para o Gateway. Ele estará listado como `NodePort` apontando para a porta 31080.

5. Implantar uma aplicação de teste e criar o HTTPRoute

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-demo
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-demo
  template:
```

```
    metadata:
      labels:
        app: web-demo
    spec:
      containers:
      - name: echo-server
        image: paulbouwer/hello-kubernetes:1.10.1
        ports:
        - containerPort: 8080
        env:
        - name: MESSAGE
          value: "Rodando com Envoy Gateway no MicroK8s!"

---
apiVersion: v1
kind: Service
metadata:
```

# Atividade Prática VII

```
  name: web-demo-svc
  namespace: default
spec:
  ports:
  - port: 80
    targetPort: 8080
  selector:
    app: web-demo

---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-demo-route
  namespace: default
spec:
  parentRefs:
```

# Atividade Prática VIII

```
- name: meu-gateway
rules:
- matches:
  - path:
      type: PathPrefix
      value: /
  backendRefs:
  - name: web-demo-svc
    port: 80
```

- Execute o comando `kubectl get deployments.apps` para verificar se a implantação da aplicação `web-demo` está ativa.

Teste a Gateway API acessando o endereço `http://localhost:31080/`

# Storage

# Persistência de dados

- Os pods do Kubernetes são efêmeros.
  - Se um Pod falha ou é destruído pelo Kubernetes, todo o seu sistema de arquivos interno é apagado.
- Para aplicações *stateful* (bancos de dados, sistemas de arquivos, filas), precisamos de persistência de dados desconectada do ciclo de vida do Pod.
- A gestão de armazenamento segue o princípio de separação de papéis entre quem administra a infraestrutura (DevOps/Admin/Cloud e quem consome o armazenamento (Desenvolvedor).
- Conceitos chave de armazenamento do Kubernetes são:
  `Volume`, `PersistentVolume`, `PersistentVolumeClaim` e `StorageClass`.

# Volume

Geral/Efêmero vs. *Semi-Persistente

- Um `Volume` é um ponto de montagem (diretório) acessível pelos contêineres dentro de um Pod. Tipos de volumes não persistentes: `emptyDir` e `hostPath`.
- `emptyDir`
  - **Como funciona:** Criado no momento em que o Pod é atribuído a um Nó. Inicialmente fica vazio. Todos os contêineres do Pod podem ler e escrever nos mesmos arquivos.
  - **Ciclo de vida:** Se o Pod for removido do nó (seja por falha ou término), os dados são apagados permanentemente.
  - **Casos de Uso:** Espaço de rascunho, diretório de cache temporário ou compartilhamento de arquivos entre contêineres de um mesmo Pod (padrão Sidecar).
- `hostPath`
  - **Como funciona:** *Monta um arquivo ou diretório do sistema de arquivos do Nó hospedeiro diretamente no Pod.
  - **Problema:** Se o Pod for destruído e recriado em outro Nó do cluster, os dados não estarão lá.
  - **Casos de Uso:** Agentes de monitoramento/logs que precisam ler componentes do nó local (ex: /var/log).

# Exemplo de Volume

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-banco-dados
  namespace: aula-k8s
spec:
  containers:
  - name: postgresql
    image: postgres:15-alpine
    env:
    - name: POSTGRES_PASSWORD
      value: "senhaSegura123"
    volumeMounts:
    - name: volume-dados-postgres
      mountPath: /var/lib/postgresql/data # Ponto de montagem dentro do contêiner
  volumes:
  - name: volume-dados-postgres
    persistentVolumeClaim:
      claimName: pvc-banco-dados # Referência ao PVC criado anteriormente
```

# PersistentVolume (PV)

- Representa um recurso real de armazenamento físico ou lógico no cluster (como um disco SSD local, um volume EBS na AWS, uma exportação NFS ou um volume Ceph).
- Diferente do `Volume` comum, o `PV` **tem ciclo de vida** totalmente **independente** de qualquer Pod.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-disco-local-01
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: armazenamento-rapido
  hostPath:
    path: "/var/discos/dados-app" # Caminho real no nó do MicroK8s
```

# PersistentVolumeClaim (PVC)

- O PVC abstrai totalmente os detalhes de infraestrutura.
- O desenvolvedor não precisa saber a senha do NFS ou a chave de API da AWS; ele apenas solicita espaço e modo de acesso.
- O PVC é o "pedido por um Persistent Volume (PV)" disponível no servidor.

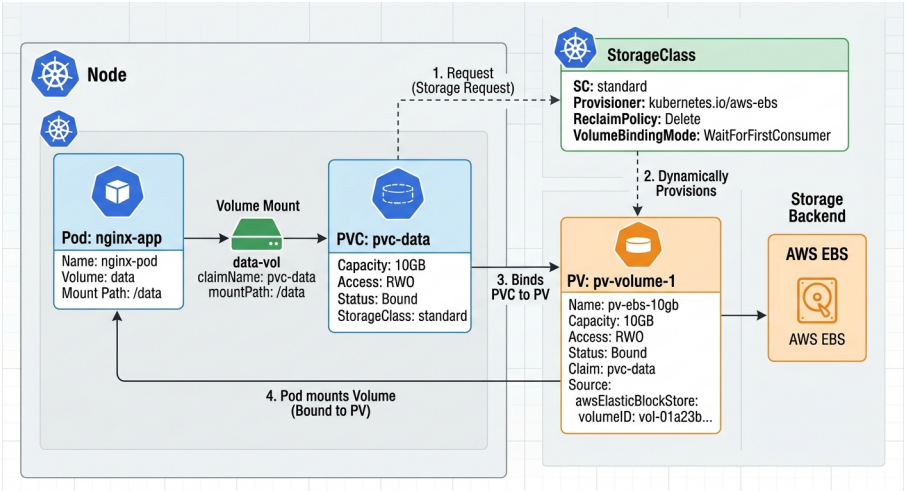
```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-banco-dados
  namespace: aula-k8s
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: armazenamento-rapido # Deve ser idêntica à SC ou PV desejado
  resources:
    requests:
      storage: 2Gi # Solicita 2GB (pode se conectar ao PV de 5GB acima)
```

# StorageClass (SC)

- A StorageClass (SC) introduz o provisionamento dinâmico. Ela cria PVs sob demanda.
- Utiliza drivers/plugins (CSI - *Container Storage Interface*) responsáveis por conversar com o provedores de infraestrutura (ex: AWS, Azure, GCP, Longhorn, Ceph, MicroK8s Hostpath).

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: armazenamento-rapido
provisioner: microk8s.io/hostpath # Provisionador local do MicroK8s
reclaimPolicy: Retain           # Mantém os dados no disco caso o PVC seja excluído
volumeBindingMode: WaitForFirstConsumer
```

# Arquitetura de Persistência do Kubernetes




Fonte: Imagem gerada com Gemini com base na documentação do Kubernetes.

# Atividade Prática

1. Aplicar os manifestos na ordem de dependência
   1.1 `$ kubectl apply -f storageclass.yaml`
   1.2 `$ kubectl apply -f pv.yaml`
   1.3 `$ kubectl apply -f pvc.yaml`
   1.4 `$ kubectl apply -f pod-volume.yaml`
2. Listar e verificar os status
   2.1 `$ kubectl get storageclass`
   2.2 `$ kubectl get pv`
   2.3 `$ kubectl get pvc -n aula-k8s`
   2.4 `$ kubectl get pod app-banco-dados -n aula-k8s`
3. Inspecionar o mapeamento completo do volume no Pod
   3.1 `$ kubectl describe pod app-banco-dados -n aula-k8s`

# Continua...

Perguntas e Discussão

# Configuration

# ConfigMap

# ConfigMap e Secrets

# Auth & Identity

# ClusterRole

# ClusterRoleBinding

# Service Account

# Atividades

# Atividades Práticas

1. Aplicar os conceitos vistos em sala de aula.

# Continua...

Perguntas e Discussão

**Prof. Alexandre Huff**
alexandrehuff@utfpr.edu.br
Universidade Tecnológica Federal do Paraná